

Stock Price Prediction Web App

Product Requirements Document & Technical Analysis

This document outlines the product requirements, recommended technology stack, system architecture, and a realistic analysis of the challenges involved in building a machine-learning-powered stock price prediction web application.

1. Problem Statement & Goal

Build a web application that allows users to select a stock, view historical price data, and see a machine-learning-generated forecast (e.g., next-day or next-N-day price/trend prediction) along with confidence levels and supporting visualizations.

It is important to frame expectations correctly from the outset: stock prices behave close to a random walk, especially over short horizons, so "prediction" should be understood as probabilistic trend/risk modeling rather than a guarantee of future prices.

Target Users

Retail investors, finance students, and hobbyist traders who want a quick, visual tool to explore trends — not professional algorithmic traders requiring millisecond-level execution.

2. Product Requirements

2.1 Functional Requirements

- User can search and select a stock by ticker symbol
- Display historical price charts (candlestick / line, configurable time range)
- Show technical indicators: moving averages, RSI, MACD, Bollinger Bands
- ML model generates price or trend predictions (e.g., next 1–7 days)
- Display predictions alongside historical data with confidence intervals
- User accounts to save watchlists and favorite stocks (optional for v1)
- Backtesting view showing how the model performed historically
- Clear disclaimer that this is not financial advice

2.2 Non-Functional Requirements

- **Data refresh:** near real-time or end-of-day, depending on API tier
- **Performance:** charts and predictions load within a few seconds
- **Scalability:** support multiple concurrent users querying different stocks
- **Security:** data-provider API keys stored server-side, never exposed to the client
- **Reliability:** graceful fallback if a data provider is unavailable

2.3 Success Metrics

- Model accuracy — directional accuracy and RMSE on price/return predictions
- App usability — time-to-prediction and page load speed
- User retention if accounts and watchlists are implemented

3. Technology Stack

Layer	Recommended Technologies	Notes
Frontend	React / Next.js + TypeScript, Tailwind CSS	Next.js gives SSR and easy deployment
Charting	TradingView Lightweight Charts, Recharts, Plotly.js	Supports candlesticks + indicator overlays
Backend	Python + FastAPI (or Flask)	FastAPI integrates cleanly with ML libraries
ML / Data Science	pandas, NumPy, scikit-learn, XGBoost/LightGBM, PyTorch/TensorFlow/Keras	Classical Models (ARIMA/Prophet) as baselines; LSTM/Transformers for advanced patterns
Feature Engineering	ta-lib or pandas-ta	Computes technical indicators efficiently
Database (time-series)	TimescaleDB (Postgres extension) or InfluxDB	Optimized for price history queries
Database (relational)	PostgreSQL	Users, watchlists, app metadata
Caching	Redis	Cache frequent quotes and predictions
Market Data APIs	yfinance (free), Alpha Vantage, Polygon.io, Finnhub.io	Free for prototyping; paid tiers for production reliability
Containerization	Docker	Consistent dev/prod environments
Hosting	Vercel/Netlify (frontend), Render/Railway/AWS (backend)	Scale independently
CI/CD	GitHub Actions	Automated testing and deployment
Monitoring	Sentry (errors), custom drift-tracking dashboard	Track prediction-vs-actual drift over time

4. System Architecture Overview

A scheduled job pulls historical and recent price data from a market data API and stores it in the time-series database. A feature engineering pipeline then computes technical indicators, and the trained model generates predictions. Predictions are cached in Redis and served via the FastAPI backend, which the React frontend queries through a REST (or GraphQL) API to render charts and forecasts.

For training, separate the system into two paths: an **offline pipeline** that retrains models periodically (e.g., weekly) on historical data, and an **online inference path** that serves predictions quickly using the latest trained model.

Data Flow Summary

- Market Data API → Time-series Database (TimescaleDB/InfluxDB)
- Database → Feature Engineering Pipeline (indicators, lagged returns)
- Features → Trained ML Model → Prediction Output
- Prediction Output → Redis Cache → FastAPI Backend
- FastAPI Backend → React/Next.js Frontend (charts & forecasts)

5. Honest Analysis: Challenges & Considerations

Model Performance Reality

Short-term stock price prediction is extremely difficult due to market efficiency and noise. Simple baselines (e.g., "tomorrow equals today") are hard to beat consistently. A more defensible product framing is predicting *direction* (up/down) or *volatility* rather than exact prices, with transparent confidence intervals and historical accuracy reporting.

Data Quality & Cost

Free APIs such as yfinance are suitable for prototyping but have rate limits and occasional reliability issues. A production app serving many users will likely need a paid data provider with SLAs.

Legal & Compliance

If predictions could influence financial decisions, the app should include clear disclaimers that it is for educational/informational purposes only and not financial advice. Depending on jurisdiction, offering investment-related predictions as a service may carry regulatory implications worth researching with a legal professional.

This document is for technical planning purposes only and does not constitute legal or financial advice.

Overfitting Risk

Deep learning models on financial time series are prone to overfitting to historical patterns that do not generalize. Robust backtesting using walk-forward validation (not a simple train/test split) is essential.

6. Suggested Development Roadmap

Phase	Focus	Outcome
Phase 1	Data pipeline + simple dashboard	Historical prices and indicators displayed; validates UI and data flow (no ML yet)
Phase 2	Baseline model	ARIMA or simple regression on lagged features, with backtesting visualization
Phase 3	Advanced models	LSTM / XGBoost models compared against the baseline
Phase 4	User features	Accounts, watchlists, UI/UX polish
Phase 5	Deployment & monitoring	Production deployment, model drift monitoring, iterative improvement